

Module 06 **Understanding Information Server Parallel Architecture**

IBM InfoSphere Information Server for Data Integration



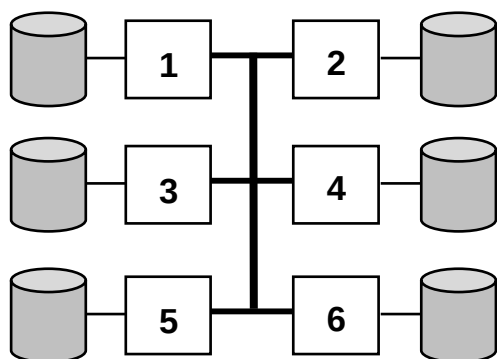
Module Objectives

After completing this topic, you should be able to:

- Describe parallel processing architecture
- Describe pipeline parallelism
- Describe partition parallelism
- List and describe partitioning and collecting algorithms
- Describe configuration files

Key Parallel Job Concepts

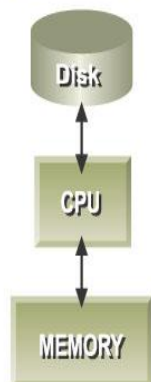
- Parallel processing:
 - Executing the job on multiple CPUs
- Scalable processing:
 - Add more resources (CPUs and disks) to increase system performance



- Example system: 6 CPUs (processing nodes) and disks
- Scale up by adding more CPUs
- Add CPUs as individual nodes or to an SMP system

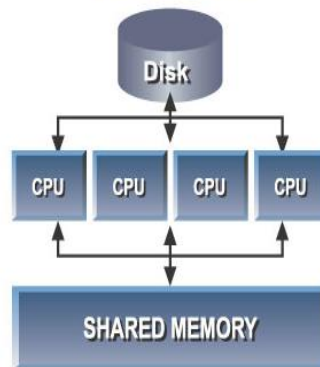
Scalable Hardware Environments

Dedicated Disk



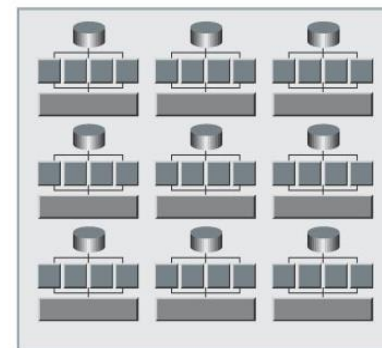
Uniprocessor

Shared Disk



SMP System
(Symmetric
Multiprocessor)

Shared Nothing



MPP, Clustered Systems
(Massively Parallel Processing)

- SMP
- Multi-CPU (2-64+)
- Shared memory & disk
- GRID / Clusters
 - Multiple, multi-CPU systems
 - Dedicated memory per node
 - Typically SAN-based shared storage
- MPP
 - Multiple nodes with dedicated memory, storage
- 2 – 1000's of CPUs

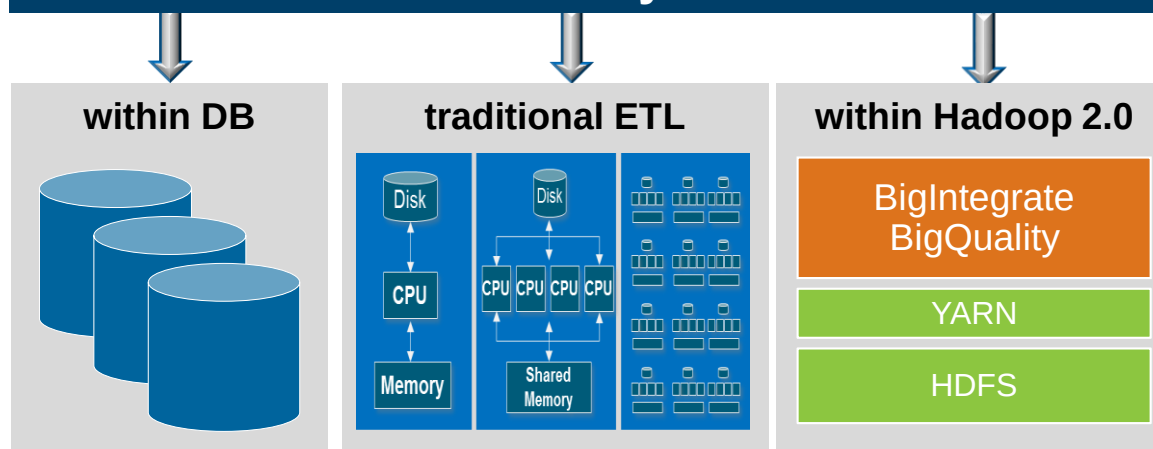
IBM's Unique Approach: Information Integration & Governance for the Enterprise



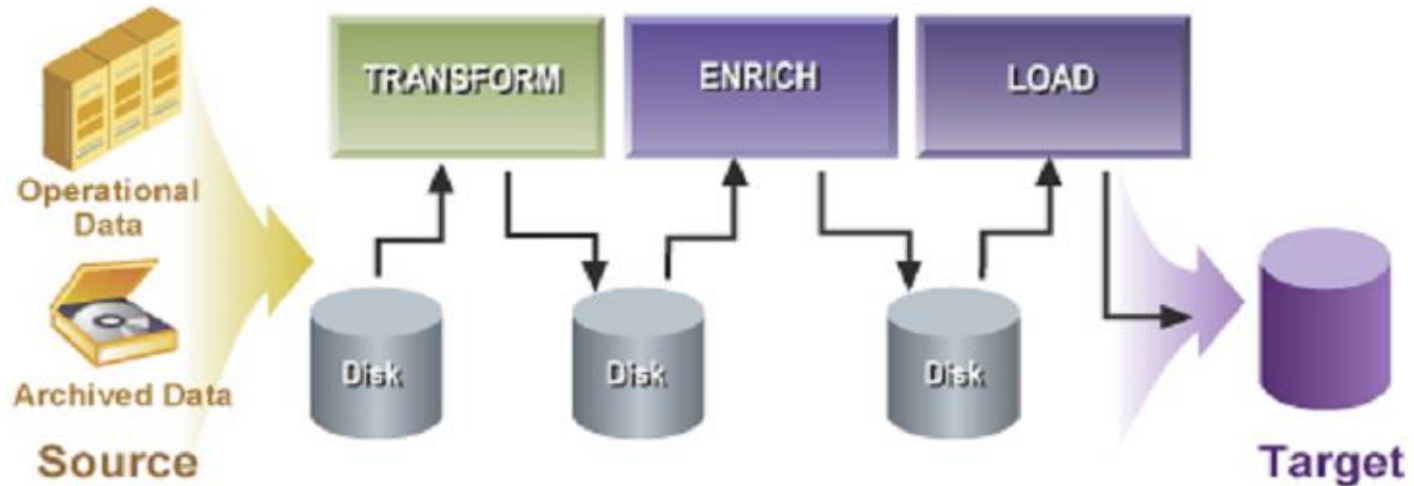
- Highest performance through native execution of Information Server engine directly on Yarn
- Design your transformation or cleansing once and run it on your Hadoop Cluster, your traditional engine or within your database

One Integration & Quality Design

Execute "Anywhere"

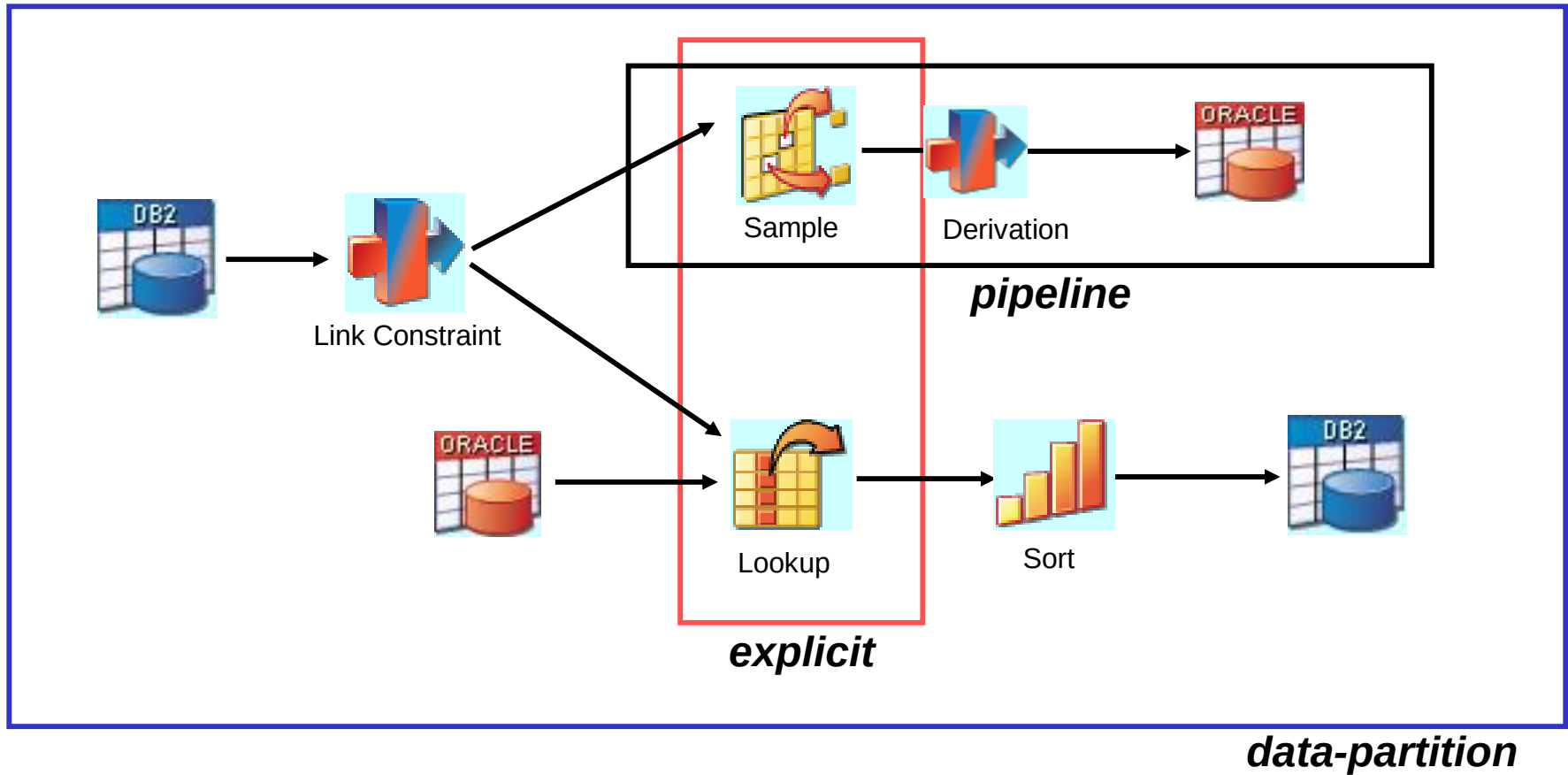


Drawbacks of Traditional Batch Processing



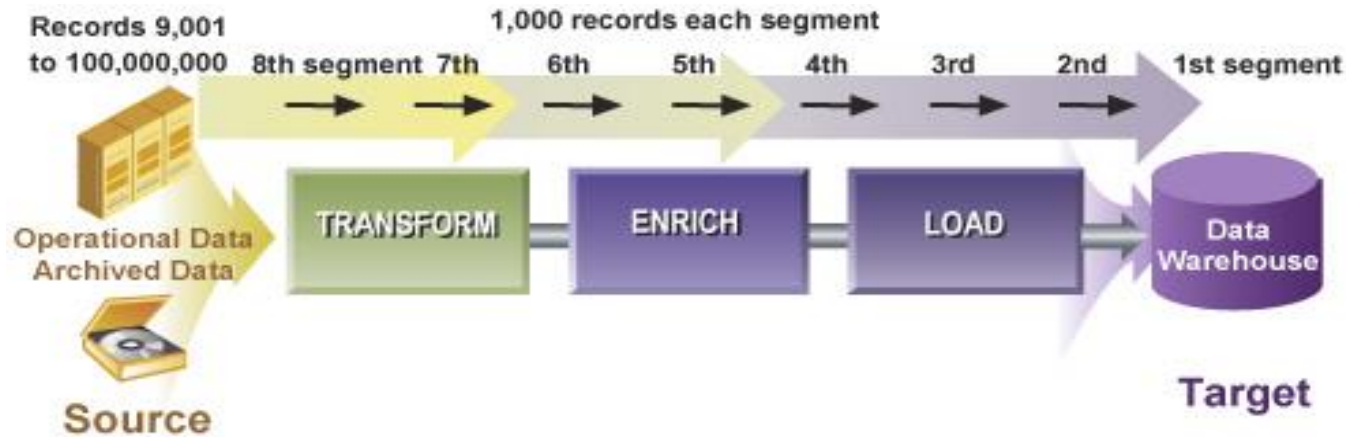
- Poor utilization of resources
 - Lots of idle processing time
 - Lots of disk and I/O for staging
- Complex to manage
- Impractical with large data volumes

Three Types of Parallelism



- Implicit pipeline parallelism
- Implicit partition parallelism
- Explicit execution flow parallelism

Pipeline Parallelism

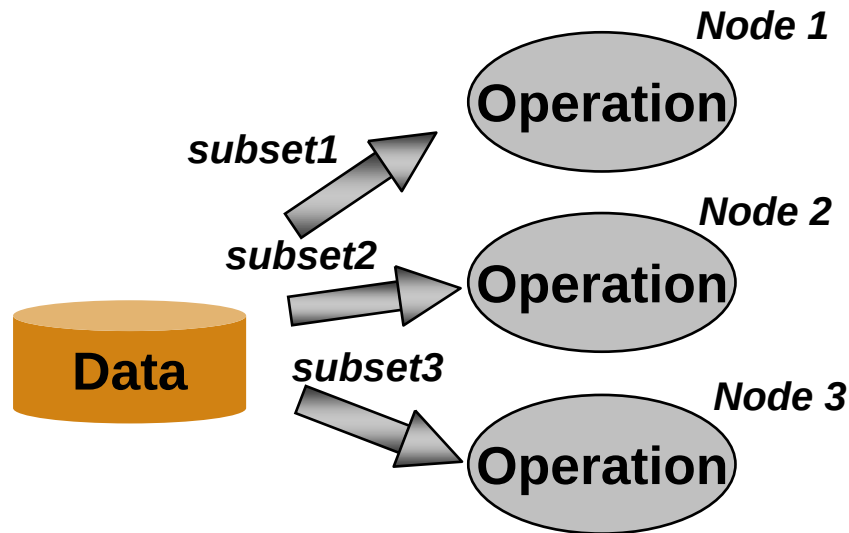


- Transform, clean, load processes overlap in execution
- Like a conveyor belt moving rows from process to process
 - Start downstream process while upstream process is running
- Advantages:
 - Reduces disk usage for staging areas
 - Keeps processors busy
- Still has limits on scalability

Partition Parallelism

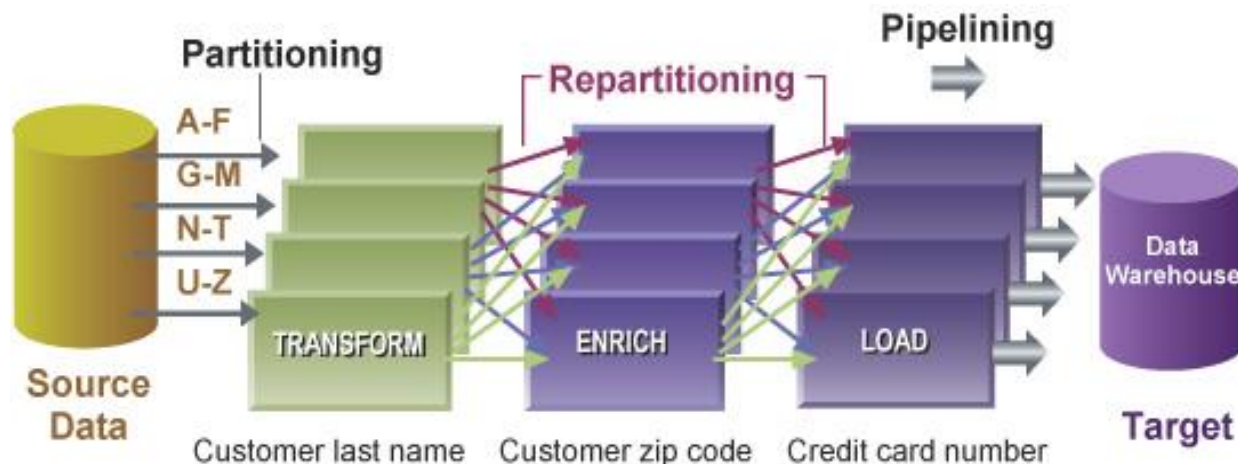
- Divide the incoming stream of data into subsets to be separately processed by an operation
 - Subsets are called partitions (nodes)
- Each partition of data is processed by the same operation
 - e.g., if operation is Filter, each partition will be filtered in exactly the same way
- Facilitates near-linear scalability
 - 8 times faster on 8 processors
 - 24 times faster on 24 processors
 - This assumes the data is evenly distributed

Three-Node Partitioning



- Here the data is partitioned into three partitions
- The operation is performed on each partition of data separately and in parallel
- If the data is evenly distributed, the data will be processed three times faster

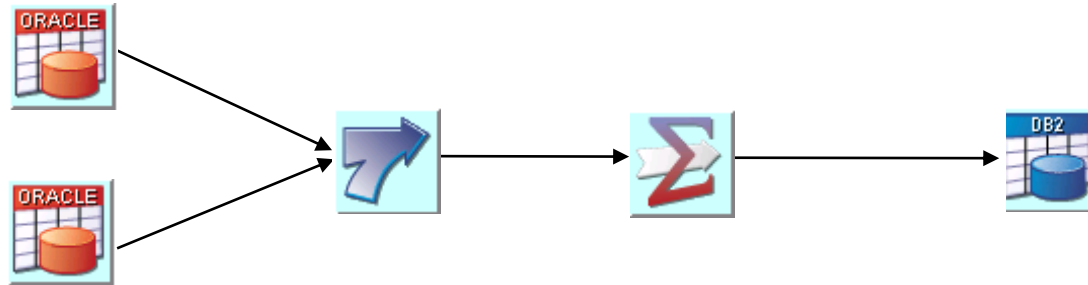
Parallel Jobs Combine Partitioning and Pipelining



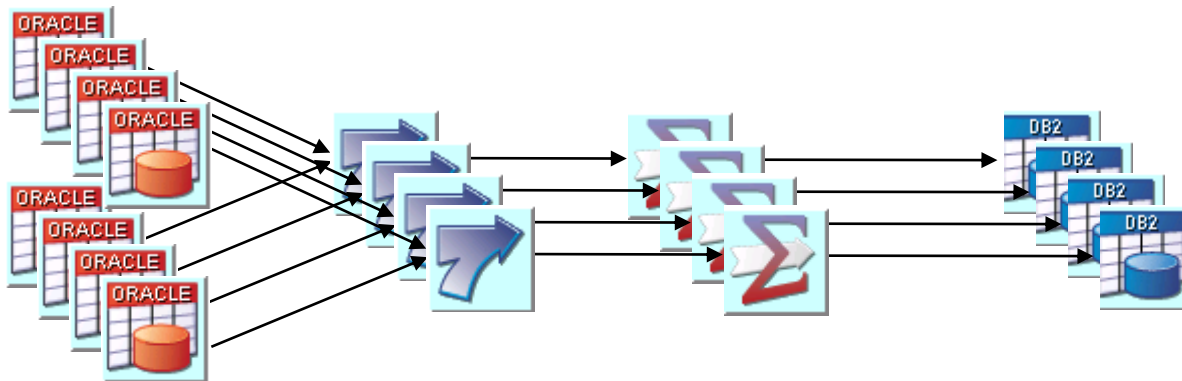
- Within parallel jobs pipelining, partitioning, and repartitioning are automatic
- Job developer only identifies:
 - **Sequential vs. parallel mode (by stage)**
 - Default for most stages is parallel mode
 - **Method of data partitioning**
 - The method to use to distribute the data into the available nodes
 - **Method of data collecting**
 - The method to use to collect the data from the nodes into a single node
 - **Configuration file**
 - Specifies the nodes and resources

Job Design and Execution

User assembles the flow using DataStage Designer

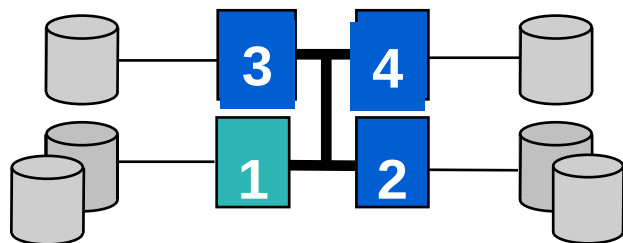


... at runtime, this job runs in parallel for any configuration
(1 node, 4 nodes, N nodes)



No need to modify or recompile the job design!

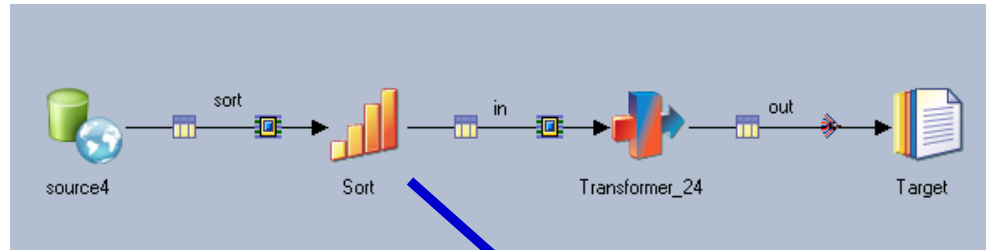
Example Configuration File



- Key points:
 - Number of nodes defined
 - Resources assigned to each node. Their order is significant.
 - Advanced resource optimizations and configuration (named pools, databases, ...)

```
{
  node "n1" {
    fastname "pseries"
    pool "" "n1" "pseries" "app2" "sort"
    resource disk "/orch/n1/d1" {}
    resource disk "/orch/n1/d2" {"bigdata"}
    resource scratchdisk "/temp" {"sort"}
  }
  node "n2" {
    fastname "pseries"
    pool "" "n2" "pseries" "app1"
    resource disk "/orch/n2/d2" {"bigdata"}
    resource disk "/orch/n2/d1" {""}
    resource scratchdisk "/temp" {}
  }
  node "n3" {
    fastname "pseries"
    pool "" "n3" "pseries" "app1"
    resource disk "/orch/n3/d1" {}
    resource scratchdisk "/temp" {}
  }
  node "n4" {
    fastname "pseries"
    pool "" "n4" "pseries" "app1"
    resource disk "/orch/n4/d1" {}
    resource scratchdisk "/temp" {}
  }
}
```

Example Configuration File



```
{
  node "node1"
  {
    fastname "dyn-9-161-124-186.mul.ie.ibm.com"
    pools ""
    resource disk "/opt/IBM/InformationServer/Server/Datasets" {pools ""}
    resource scratchdisk "/opt/IBM/InformationServer/Server/Scratch" {pools ""}
  }
  node "node2"
  {
    fastname "dyn-9-161-124-186.mul.ie.ibm.com"
    pools "" "group1" "group2"
    resource disk "/opt/IBM/InformationServer/Server/Datasets" {pools ""}
    resource scratchdisk "/opt/IBM/InformationServer/Server/Scratch" {pools ""}
  }
  node "node3"
  {
    fastname "dyn-9-161-124-186.mul.ie.ibm.com"
    pools "" "group1"
    resource disk "/opt/IBM/InformationServer/Server/Datasets" {pools ""}
    resource scratchdisk "/opt/IBM/InformationServer/Server/Scratch" {pools ""}
  }
}
```

Sort - Sort

Stage | Input | Output

Stage name:
Sort

General | Properties | Advanced | NLS Locale

Execution mode: Default (Parallel) Combinability mode: (Auto) Preserve partitioning: Default (Set)

Configuration file:
default2

☒ Node pool and resource constraints:

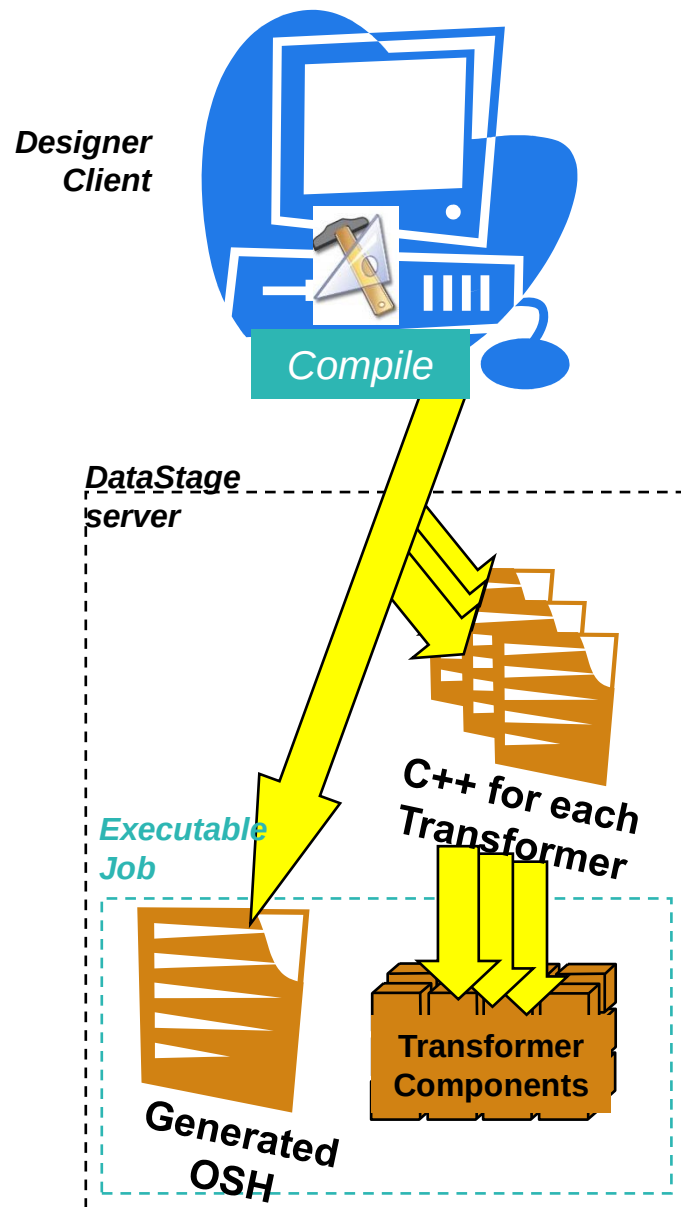
	Constraint	Type	Name
1	Node pool	<Not applicable>	group1

☐ Node map constraint:

OK Cancel Help

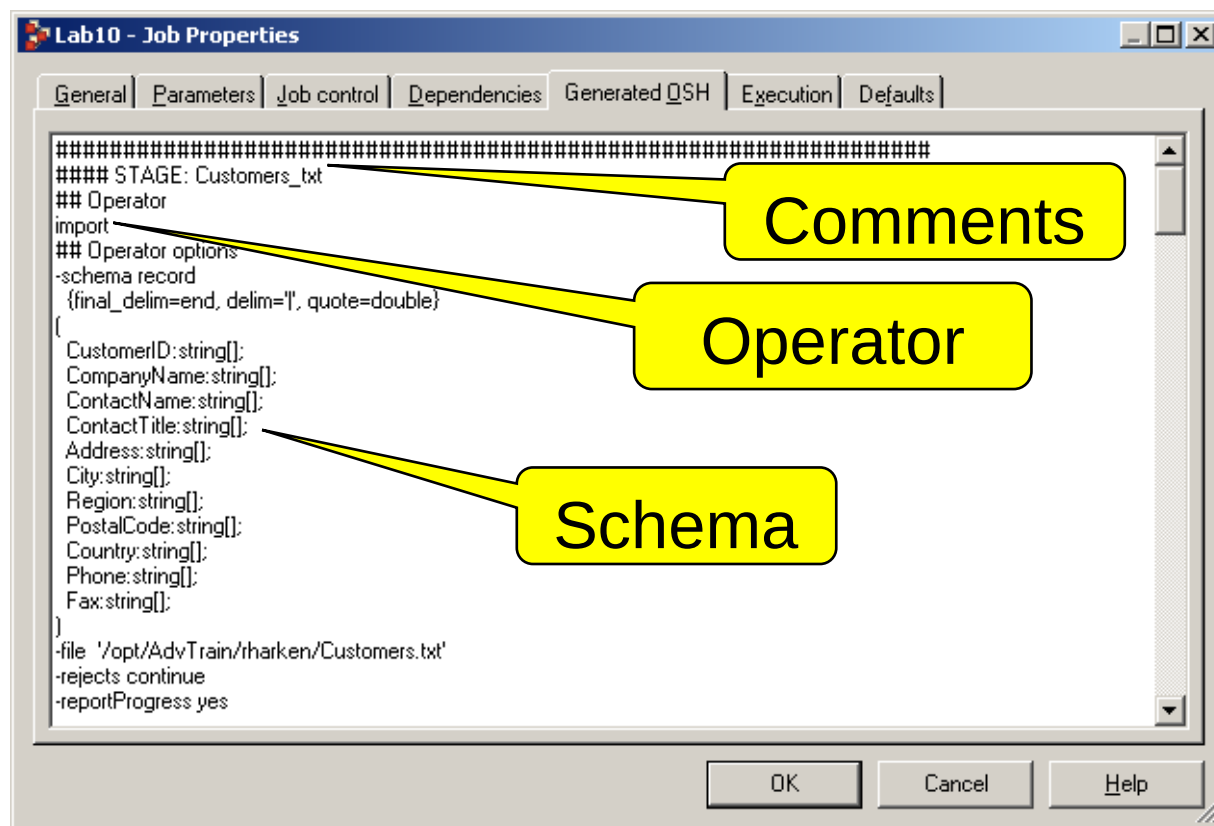
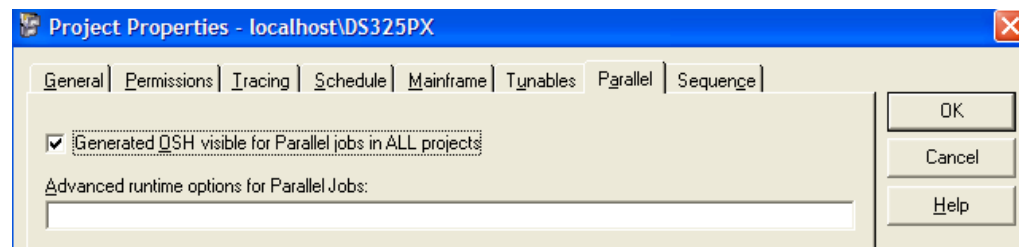
Parallel Job Compilation

- What gets generated
 - OSH: a kind of script
 - OSH represents the design data flow and stages
 - Stages become operators
- A transform operator for each Transformer
 - A custom operator is built during the compile
 - Compiled into C++ then to corresponding native object module
 - A C++ compiler is needed for jobs with Transformers



Generated OSH

Enable viewing of generated OSH in Administrator:



OSH is visible in:

- Job properties
- Job run log

Stage to Operator Mapping Examples

- Sequential File
 - Source: import
 - Target: export
- Data Set: copy
- Sort: tsort
- Aggregator: group
- Row Generator, Column Generator, Surrogate Key Generator: generator
- Oracle
 - Source: oraread
 - Sparse lookup: oralookup
 - Target load: orawrite

Partitioning and Collecting

- Partitioning breaks incoming rows into multiple streams of **rows** (one for each node)
- Each partition of rows is processed separately by the stage/operator
- Collecting returns partitioned data back to a single stream
- Partitioning / Collecting is specified on stage input links

Partitioning Algorithms

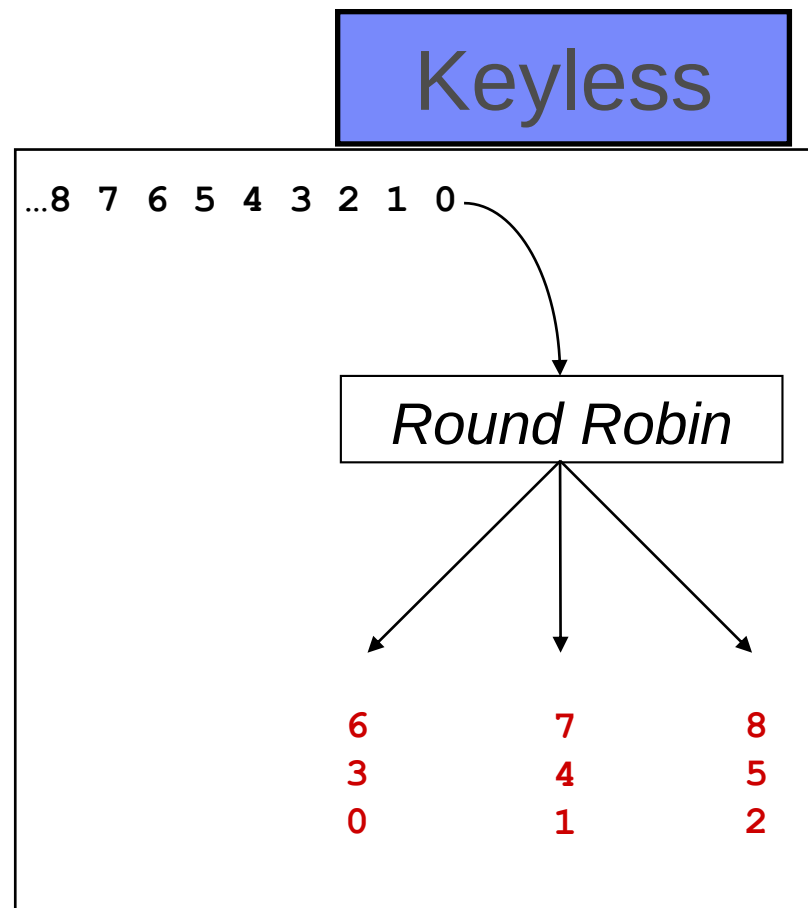
- Partitioning algorithms include:
 - Round robin
 - Random
 - Hash: Determine partition based on key value
 - Requires key specification
 - DB2
 - Modulus
 - Entire: Send all rows down all partitions
 - Same: Preserve the same partitioning
 - **Auto**: Let DataStage choose the algorithm

Keyless vs. Keyed Partitioning Algorithms

- **Keyless:** Rows are distributed independently of data values
 - Round Robin
 - Random
 - Entire
 - Same
- **Keyed:** Rows are distributed based on values in the specified key
 - Hash: Partition based on key
 - Example: Key is State. All “CA” rows go into the same partition; all “MA” rows go in the same partition. Two rows of the same state never go into different partitions
 - Modulus: Partition based on modulus of key divided by the number of partitions. Key is a numeric type.
 - Example: Key is OrderNumber (numeric type). Rows with the same order number will all go into the same partition.
- DB2: Matches DB2 DPF partitioning

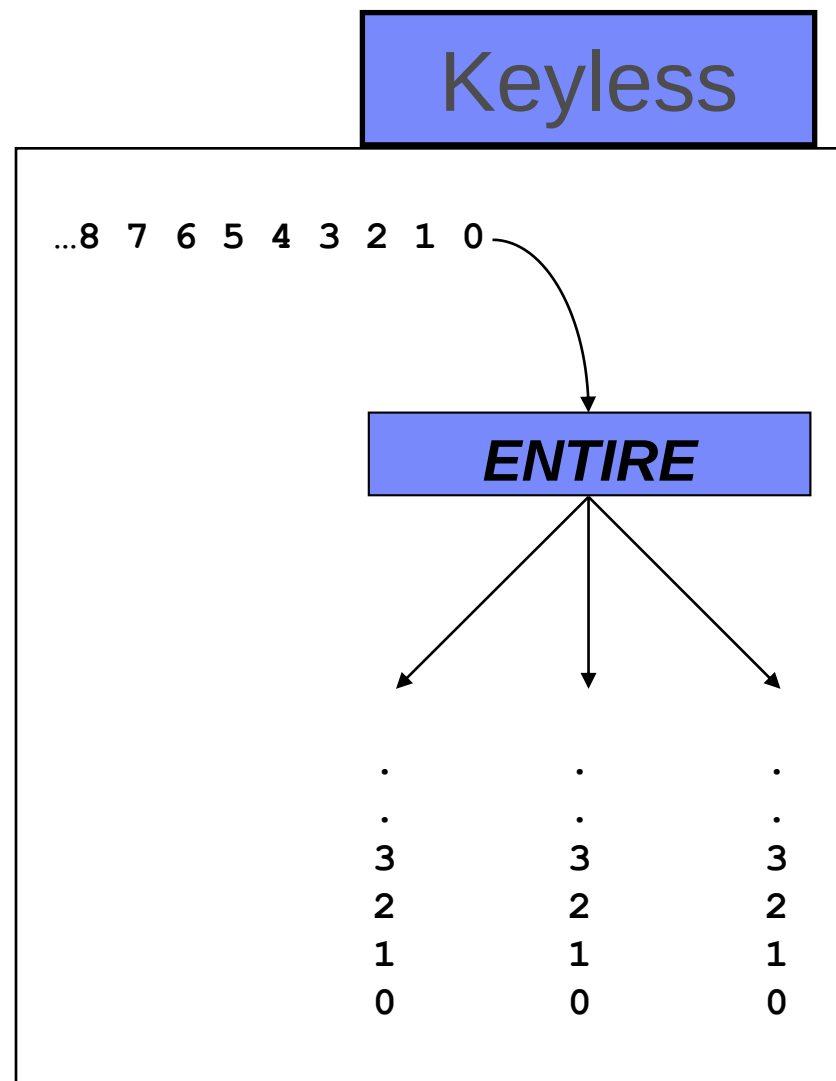
Round Robin and Random Partitioning

- Keyless partitioning methods
- Rows are evenly distributed across partitions
 - Good for initial import of data if no other partitioning is needed
 - Useful for redistributing data
- Fairly low overhead
- Round Robin assigns rows to partitions like dealing cards
 - Row/Partition assignment will be the same for a given `$APT_CONFIG_FILE`
- Random has slightly higher overhead, but assigns rows in a non-deterministic fashion between job runs



ENTIRE Partitioning

- Each partition gets a complete copy of the data
 - Useful for distributing lookup and reference data
 - May have performance impact in MPP / clustered environments
 - On SMP platforms, Lookup stage (only) uses shared memory instead of duplicating ENTIRE reference data
 - On MPP platforms, each server uses shared memory for a single local copy
- ENTIRE is the default partitioning for Lookup reference links with “Auto” partitioning
 - On SMP platforms, it is a good practice to set this explicitly on the Normal Lookup reference link(s)



HASH Partitioning

- Keyed partitioning method
- Rows are distributed according to the values in key columns
 - Guarantees that rows with same key values go into the same partition
 - Needed to prevent matching rows from “hiding” in other partitions
 - e.g. Join, Merge, Remove Duplicates, ...
 - Partition distribution is relatively equal if the data across the source key column(s) is evenly distributed

Keyed

Values of key column

...0 3 2 1 0 2 3 2 1 1

HASH

0	1	2
3	1	2
0	1	2
3		

Modulus Partitioning

- Keyed partitioning method
- Rows are distributed according to the values in one integer key column
 - Uses modulus
$$\text{partition} = \text{MOD}(\text{key_value} / \text{\#partitions})$$
- Faster than HASH
- Guarantees that rows with identical key values go to the same partition
- Partition size is relatively equal if the data within the key column is evenly distributed
- This method is similar to hash by field, but involves simpler computation.

Keyed

Values of key column

...0 3 2 1 0 2 3 2 1 1

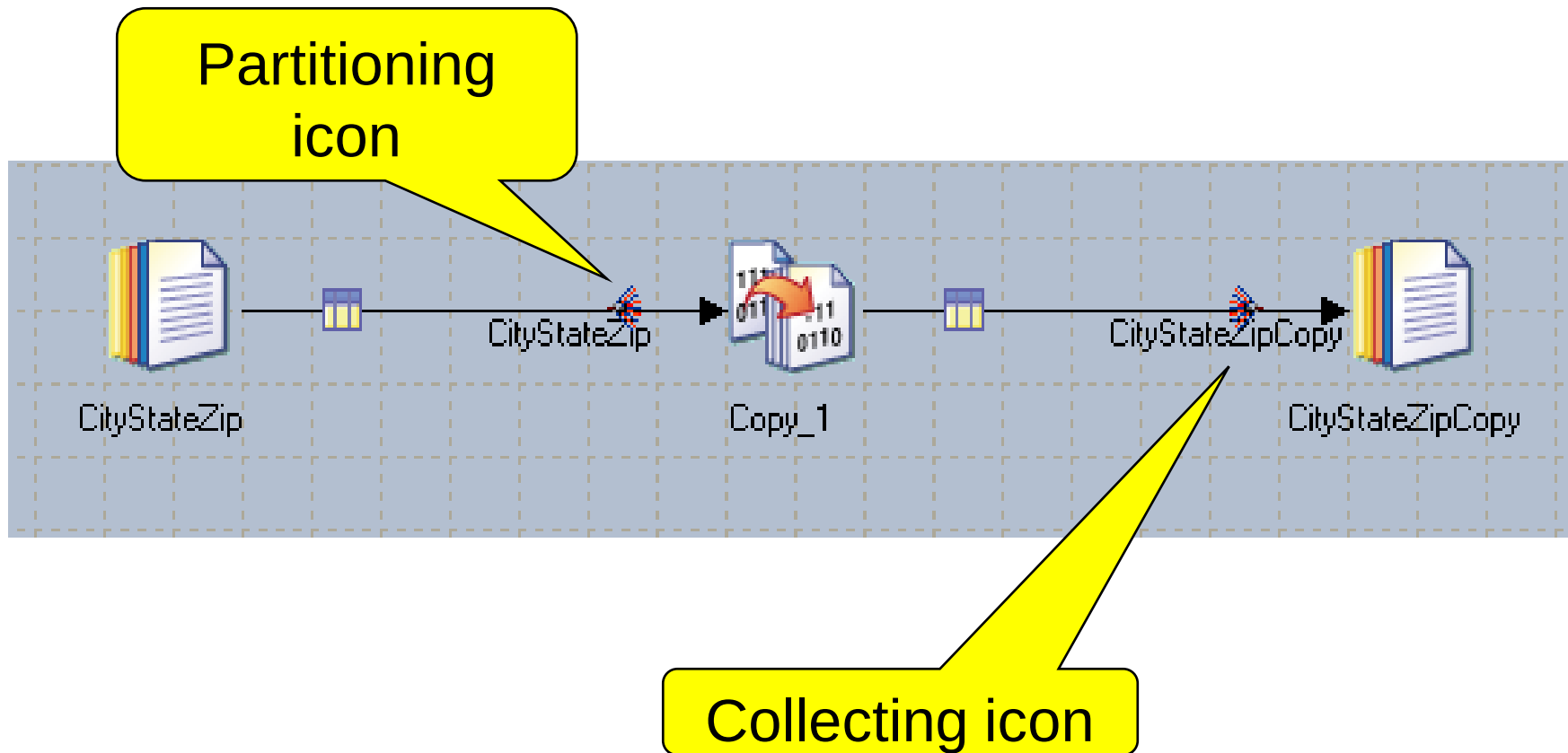
MODULUS

0	1	2
3	1	2
0	1	2
3		

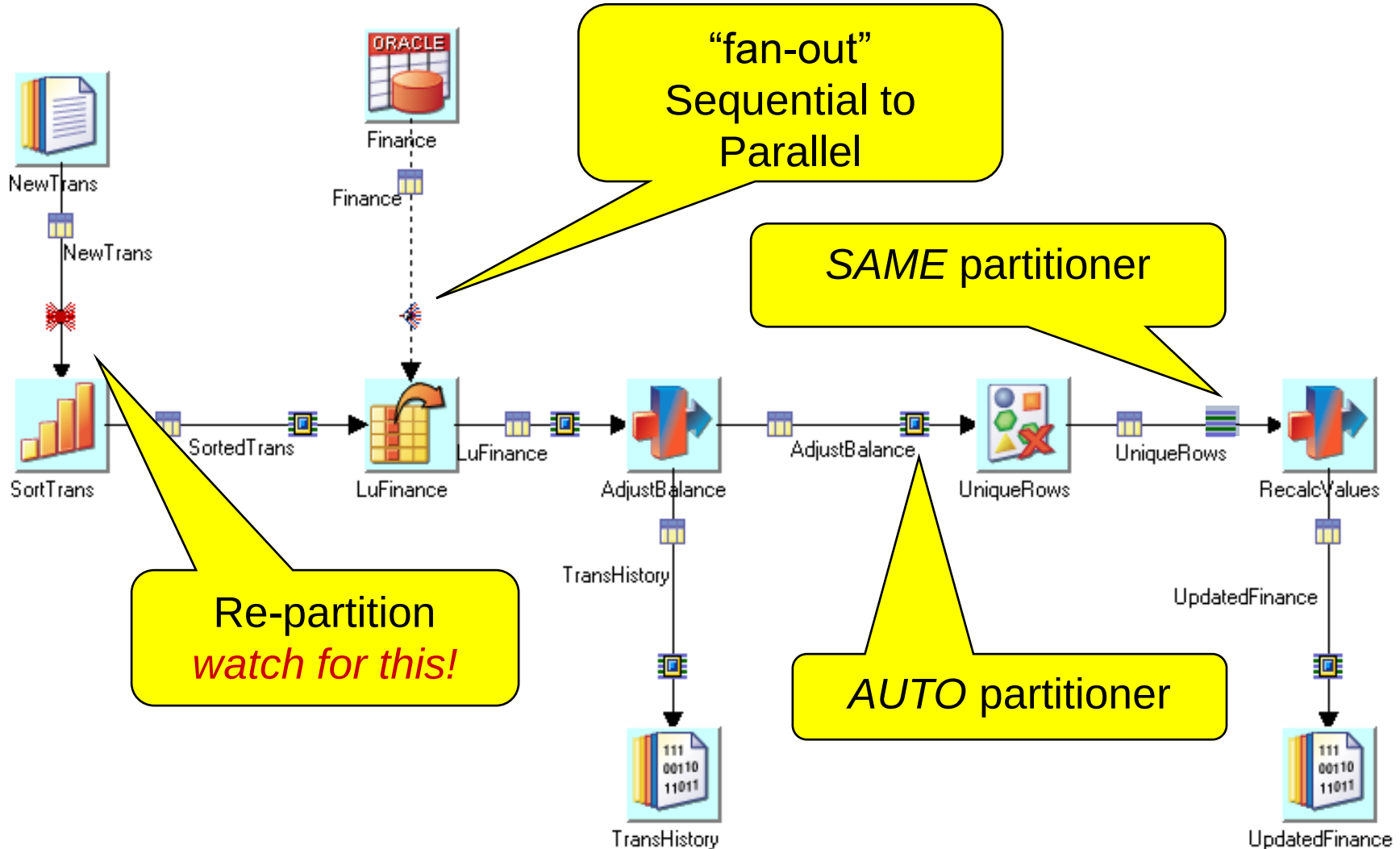
Auto Partitioning

- DataStage inserts partition components as necessary to ensure correct results
 - Before any stage with “Auto” partitioning
 - Generally chooses ROUND-ROBIN or SAME
 - Inserts HASH on stages that require matched key values (e.g. Join, Merge, Remove Duplicates)
 - Inserts ENTIRE on Normal (not Sparse) Lookup reference links
 - NOT always appropriate for MPP/clusters
- Since DataStage has limited awareness of your data and business rules, explicitly specify HASH partitioning when needed
 - DataStage has no visibility into Transformer logic
 - Hash is required before Sort and Aggregator stages
 - DataStage sometimes inserts un-needed partitioning
 - Check the log

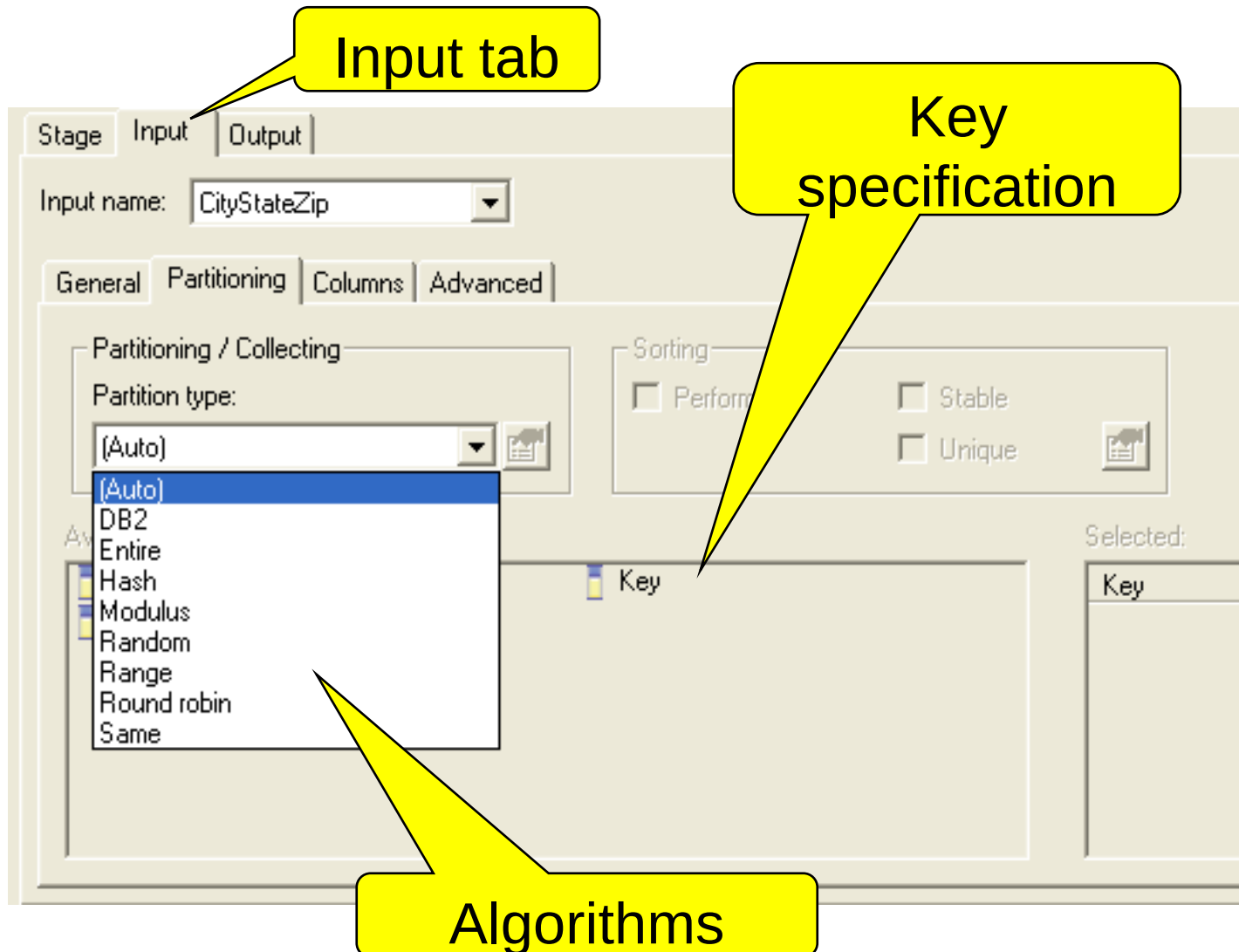
Partitioning / Collecting Link Icons



More Partitioning Icons



Partitioning Tab



Collecting Algorithms

- Collecting algorithms include:
 - Round robin
 - Auto
 - Collect first available record
 - Sort Merge
 - Read in by key
 - Presumes data is sorted by the key in each partition
 - Builds a single sorted stream based on the key
 - Ordered
 - Read all records from first partition, then second, ...

Just as for partitioning, in many situations you can leave DataStage® to work out the best collecting method to use.

Collecting Specification

